

DOCUMENTACIÓN TÉCNICA OFICIAL

MANUAL TÉCNICO Y DE ARQUITECTURA DE SOFTWARE

Plataforma Multijugador en Tiempo Real JuegoClub (UNO & Dominó)

ASP.NET Core Web API

SignalR WebSockets

Angular 17

Entity Framework Core

Capacitor 5

Docker

Proyecto: JuegoClub - UnoyDomino • **Versión:** 1.0.0

Autores: Marvin Campos Exequiel y Keyron Fernández Freckleton

1. Ficha Técnica del Sistema

El sistema **JuegoClub (UnoyDomino)** es una plataforma de entretenimiento multijugador diseñada para ejecutar partidas en tiempo real de juegos de mesa tradicionales (UNO y Dominó Double-Six). Cuenta con soporte para emparejamiento automático, salas privadas mediante códigos de 6 caracteres, integración de bots con IA estratégica, sistema económico de monedas/experiencia, tienda de cosméticos y soporte híbrido web/móvil.

Componente	Tecnología / Especificación
Framework Backend	ASP.NET Core Web API (.NET 10 / net8.0 compatible)
Protocolo Tiempo Real	ASP.NET Core SignalR vía WebSockets (`/gamehub`)
Framework Frontend	Angular 17 (Standalone Components, Signals, RxJS, TailwindCSS)
Compilación Móvil	Capacitor 5 (Android Studio Build & Gradle)
Persistencia & ORM	Entity Framework Core 8/10 (SQL Server / PostgreSQL / MySQL / InMemory)
Autenticación & Seguridad	Cifrado Hash + Salt HMACSHA512 para contraseñas de usuarios
Contenedores	Dockerfile multi-stage build listo para producción

Resumen de Capacidades

El sistema soporta partidas simultáneas sin interferencia mediante aislamiento por grupos en SignalR (`codigoSala`), estado sincronizado de cartas y fichas en memoria con persistencia diferida en base de datos.

2. Arquitectura de Software N-Capas

La solución está estructurada bajo el patrón de arquitectura N-Capas para garantizar un desacoplamiento limpio entre la representación de datos, reglas de negocio y los controladores de entrada/salida.



Desglose de Capas del Backend

- **JuegoClub.Dominio:** Contiene las entidades persistentes (`Usuario`, `Articulo`, `Logro`, `HistorialPartida`), enumeradores (`TipoJuego`, `ResultadoPartida`) y clases de estado volátil de partida (`Sala`, `EstadoUno`, `EstadoDomino`).
- **JuegoClub.AccesoDatos:** Define el `JuegoClubContext` para Entity Framework Core con inicialización de modelos y sembrado automático de datos iniciales.
- **JuegoClub.LogicaNegocios:** Aloja los motores de cómputo `MotorUnoService`, `MotorDominoService`, `AuthService`, `EconomiaService`, `ExperienciaService` y `LogroService`.
- **JuegoClub.API:** Capa expuesta con controladores HTTP REST y el hub de WebSockets `GameHub.cs`.

3. Estructura del Código Fuente

```
UnoyDomino/
├── JuegoClub.Dominio/           # Capa de Dominio (Entidades, Clases de Estado, DTOs)
│   ├── Entidades/              # Usuario.cs, Articulo.cs, Logro.cs, HistorialPartida.cs
│   └── Clases/                 # Sala.cs, EstadoUno.cs, EstadoDomino.cs
├── JuegoClub.AccesoDatos/       # Capa de Persistencia (EF Core DbContext)
│   └── Contexto/               # JuegoClubContext.cs
├── JuegoClub.LogicaNegocios/    # Capa de Lógica de Negocio (Motores y Servicios)
│   └── Implementaciones/       # MotorUnoService.cs, MotorDominoService.cs, AuthService.cs...
├── JuegoClub.API/              # Capa de Presentación REST & SignalR
│   ├── Controllers/           # AuthController.cs, TiendaController.cs, PerfilController.cs...
│   └── Hubs/                  # GameHub.cs (Canal multijugador en tiempo real)
└── JuegoClub.UI/               # Cliente Angular 17 SPA & Capacitor Android
    ├── src/app/Components/    # auth, dashboard, lobby, uno-table, domino-table...
    └── src/app/Services/      # game.service.ts (SignalR Client Wrapper)
```

4. Modelo de Datos (Base de Datos)

Tabla	Campos Clave	Descripción / Propósito
Usuarios	`Id` (PK), `Username`, `Email`, `PasswordHash`, `PasswordSalt`, `Monedas`, `Nivel`, `ExperienciaActual`	Almacena el perfil del usuario, credenciales cifradas y saldo de economía.
Articulos	`Id` (PK), `Nombre`, `Descripcion`, `Tipo`, `Precio`, `Valor`	Catálogo cosmético de la tienda (Bordes de Avatar, Aspectos de Tablero, Sonidos).
Logros	`Id` (PK), `Titulo`, `Descripcion`, `PuntosExperienciaRecompensa`	Catálogo de metas desbloqueables con recompensas de XP.
HistorialPartidas	`Id` (PK), `UsuarioId` (FK), `Juego`, `Resultado`, `MonedasApostadas`, `MonedasGanadasOPerdidas`	Registro auditable de partidas jugadas con fecha y balance financiero.

5. Endpoints API REST y Contratos SignalR Hub

5.1. Endpoints REST (`/api`)

Método	Ruta Endpoint	Descripción
POST	/api/auth/register	Crea una nueva cuenta de usuario validando duplicados y generando Hash+Salt.
POST	/api/auth/login	Autentica credenciales y retorna el objeto de usuario y estado.
GET	/api/perfil/{id}	Retorna el resumen del perfil, inventarios y estadísticas.
GET	/api/tienda/articulos	Obtiene la lista completa de artículos disponibles en la tienda.
POST	/api/tienda/comprar	Procesa la compra deduciendo monedas y registrando la propiedad del artículo.

5.2. Eventos SignalR (`/gamehub`)

El intercambio de datos en tiempo real entre cliente y servidor se realiza mediante los siguientes eventos:

- **CrearSala / UnirseSala:** Gestiona la asignación del cliente a la sala y emite `ActualizarEstadoSala`.
- **JugarContraBots:** Inicializa la mesa y añade instancias de IA para completar el cupo de jugadores.
- **JugarCarta (UNO) / ColocarFicha (Dominó):** Procesa la jugada en el motor correspondiente y transmite `ActualizarEstadoUno` o `ActualizarEstadoDomino` a todos los integrantes de la sala.
- **EnviarEmote / EnviarMensajeChat:** Difunde mensajes y reacciones interactivas en pantalla.

6. Reglas de Negocio y Motores de Juego

6.1. Motor UNO (`MotorUnoService`)

- **Mazo y Reparto:** Cada jugador inicia con 7 cartas al azar. El mazo de descarte se inicializa con una carta numérica.
- **Validación de Jugadas:** Una carta es válida si coincide en color, número o símbolo con la carta superior del mazo de descarte, o si es un comodín.
- **Efectos Especiales:** Cartas Toma 2 (`+2`), Comodín Toma 4 (`+4`), Cambio de Color, Reversa y Salto procesan la rotación de turnos y acumulación de penalizaciones.
- **Inteligencia de Bots:** Los bots analizan su mano para lanzar la carta de mayor valor estratégico o robar del mazo automáticamente.

6.2. Motor Dominó (`MotorDominoService`)

- **Juego Doble-Seis:** Juego tradicional con 28 fichas (desde `0-0` hasta `6-6`). Se reparten 7 fichas por jugador.
- **Mesa Extremos:** Mantiene la referencia visual del extremo izquierdo y derecho del tablero.
- **Pozo (Sopa):** **** Permite robar fichas si el jugador en turno no posee fichas jugables en sus extremos.**
- **Traba / Cierre:** Detecta cuándo ningún jugador puede colocar ficha, sumando los puntos restantes en mano para definir al ganador por menor puntaje.

7. Guía de Instalación, Ejecución y Despliegue

Ejecución Local paso a paso

```
# 1. Ejecutar Backend .NET API
cd UnoyDomino
dotnet run --project JuegoClub.API

# El servidor iniciará escuchando en http://localhost:5159/

# 2. Ejecutar Frontend Angular UI (en otra terminal)
cd UnoyDomino/JuegoClub.UI
npm install
npm start

# El cliente web iniciará en http://localhost:4200/
```

Despliegue con Docker

```
# Compilar la imagen Docker del backend
docker build -t juegoclub-api -f UnoyDomino/Dockerfile .

# Ejecutar el contenedor
docker run -d -p 5159:5159 --name juegoclub_backend juegoclub-api
```

Compilación Móvil Android (Capacitor)

```
cd UnoyDomino/JuegoClub.UI
ng build --configuration production
npx cap sync android
npx cap open android
```

8. Mantenimiento y Escalabilidad

Para escalar la aplicación horizontalmente tras un balanceador de carga en entornos de alta concurrencia:

1. Instalar el paquete `Microsoft.AspNetCore.SignalR.StackExchangeRedis` en la API.
2. Configurar el Backplane de Redis en `Program.cs` llamando a `builder.Services.AddSignalR().AddStackExchangeRedis(...)`.
3. Esto sincronizará los estados de salas y eventos de WebSockets entre múltiples servidores de backend sin pérdida de sesión.

Fin del Manual Técnico • JuegoClub UnoyDomino Version 1.0.0